

전자전기컴퓨터설계실험II

실험 후 보고서

LAB7

학과	전자전기컴퓨터공학부
이름 / 학번	김승민 / 2023440021
작성일	2026년 11월 6일
대상	SPI 통신과 DAC, SERVO 제어
검증 환경	VS Code / Vivado

1. 실험 목적과 관련 이론

실험 목적

SPI 과제 8~12를 수행하여 MCU와 FPGA의 송신, 수신 조합별 데이터 전달을 확인하였다. 또한 LAB6에서 완료하지 못했던 DAC와 SERVO 실험을 이어서 수행하였다. 최종적으로 다섯 SPI 과제와 DAC, SERVO의 보드 동작을 모두 확인하였다.

과제 8의 오실로스코프 파형을 기준으로 비트 전송을 해석하고, 과제 9~12에서는 수신 UART 로그를 송신 패턴과 비교하였다.

SPI 통신 원리

SPI는 마스터가 생성하는 SCK에 맞추어 데이터를 전달하는 동기식 직렬 통신이다. 이번 실험은 Mode 0, 8비트, MSB 우선으로 설정하였다. SCK는 대기 중 LOW이고 수신기는 상승 에지에서 MOSI를 읽는다. 첫 비트는 첫 상승 에지 전에 준비하고 이후 비트는 하강 에지를 기준으로 갱신한다.

CS가 LOW인 구간의 상승 에지 8번이 한 바이트를 구성한다. 이번 실험은 단방향 전송이므로 SCK, MOSI, CS와 공통 GND를 사용하고 MISO는 연결하지 않았다. 수신한 바이트는 UART를 통해 PC에 16진수로 출력하였다.

항목	설정
SPI 모드	Mode 0, 8비트, MSB 우선
검증 데이터	55 → A5 → 00 → FF 반복
MCU 송신 SCK	설계값 781.25 kHz
FPGA 송신 SCK	설계값 500 kHz
UART 모니터	9600 bps, 8N1
설계 도구	VS Code, Vivado

보드 배선과 실행 기준

MCU는 NUCLEO-G474RE를 사용하였다. SPI1 SCK는 PB3(D3), MOSI는 PA7(D11)이다. 송신 CS는 PB6(D10)이고, MCU가 수신할 때는 하드웨어 NSS 입력 PA4(A2)를 사용한다. D13의 SCK 표시는 이번 코드의 PB3 배선과 구분해야 한다.

FPGA 신호	보드 단자	패키지 핀
SCK	PORT A I/O 00	F8
MOSI	PORT A I/O 01	E8
CS	PORT A I/O 02	A7
UART TX	USB to Serial	F6

FPGA 핀은 보드의 BDIO 핀 구성과 XDC를 대응시켰다. PORT A ENABLE은 OFF로 두어 활성 LOW 버스 스위치를 연결하고, 장치 사이 GND를 공통으로 연결하였다. 송신 FPGA는 spi_tx_top, 수신 FPGA는 spi_rx_top을 각각 최상위 모듈로 설정하였다.

2. 과제 8 MCU 송신 파형 확인

설계 목표와 회로 구성

MCU에서 네 가지 바이트를 보내고, 오실로스코프 CH1을 SCK(PB3), CH2를 MOSI(PA7)에 연결하였다. 프로브 접지는 MCU GND에 연결하였다. 저장된 사진은 SCK와 MOSI 두 신호의 파형이다.

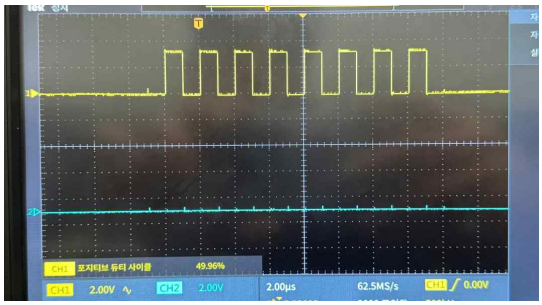
송신 핵심 코드

```
HAL_GPIO_WritePin(
 SPI_CS_ece2_GPIO_Port,
 SPI_CS_ece2_Pin, GPIO_PIN_RESET);
HAL_StatusTypeDef result =
 HAL_SPI_Transmit(
  &hspi1, &tx, 1U, 100U);
HAL_GPIO_WritePin(
 SPI_CS_ece2_GPIO_Port,
 SPI_CS_ece2_Pin, GPIO_PIN_SET);
```

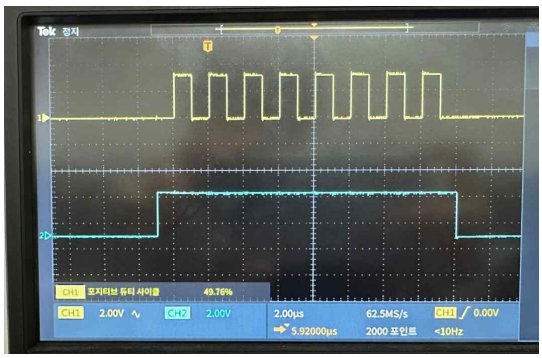
송신 전 CS를 LOW로 만들고, 1바이트 송신 후 HIGH로 복귀시킨다. HAL_SPI_Transmit의 100U는 대기 제한 시간 100 ms이다. result=0은 HAL_OK이며, 상대 장치의 수신 성공은 별도의 수신 로그로 확인해야 한다.

실제 파형과 데이터 해석

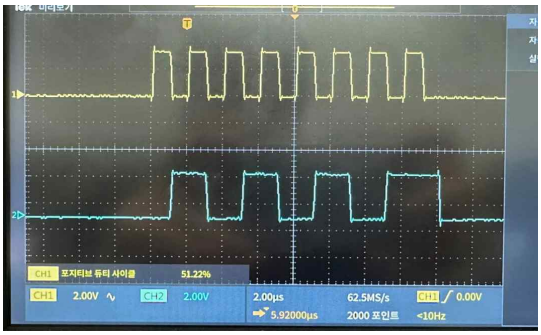
송신값	상승 에지에서 읽은 비트
0x00	00000000
0xFF	11111111
0x55	01010101
0xA5	10100101



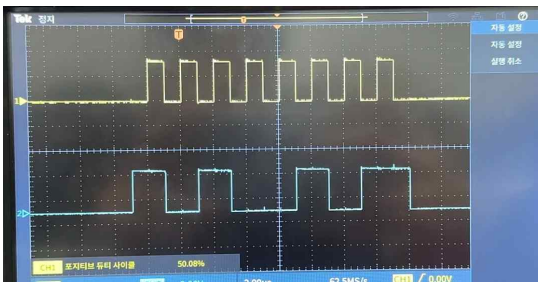
- 0x00 전송 시 MOSI가 LOW를 유지한다.



- 0xFF 전송 시 8개 샘플 구간이 HIGH



- 0x55의 비트열은 01010101이다.



- 0xA5의 비트열은 10100101이다.

각 사진에서 노란 SCK의 상승 에지는 8번이고, 해당 시점의 청록색 MOSI 값을 왼쪽부터 읽으면 표의 비트열과 일치한다. 0x55는 LOW와 HIGH가 교대로 나타나고, 0xA5는 앞부분 1010과 뒷부분 0101이 이어져 비트 순서를 확인할 수 있다.

실험 영상과 사진 모두 Github에 업로드하였다.
https://github.com/thechef030/STM32_3/tree/stm32-3/evidence/board

예상과 실제 결과 비교

네 가지 데이터가 모두 예상 비트열과 일치하여 8비트 MSB 우선 송신을 확인하였다. SCK의 대기 상태도 LOW로 나타났다. MCU 설정으로 계산한 SCK는 100 MHz/128=781.25 kHz이고 주기는 1.28 μs이다. 사진의 자동 주파수 표시는 짧은 버스트와 긴 대기 구간 때문에 서로 다르게 나타나므로 그 표시만으로 SCK 주파수를 확정하지 않았다. 본 파형에서는 8클록과 데이터 패턴의 일치 여부를 중심으로 판정하였다.

3. 과제 9 MCU 간 SPI 통신

설계 목표와 회로 구성

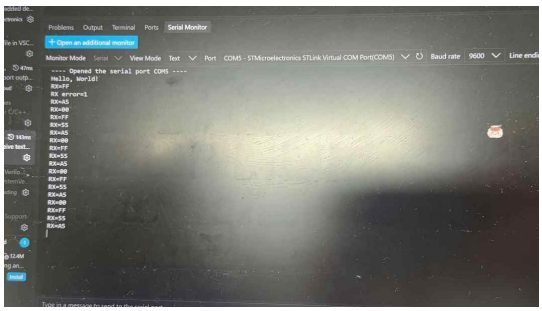
MCU 두 대를 마스터 송신기와 슬레이브 수신기로 구성하였다. 송신기의 D3와 D11을 수신기의 D3와 D11에 연결하고, 송신 D10을 수신 A2에 연결하였다. 두 MCU의 GND도 연결하였다. 수신 MCU의 ST-LINK VCP를 PC의 시리얼 모니터에서 열었다.

수신 핵심 코드

```
uint8_t rx = 0;
HAL_StatusTypeDef result =
 HAL_SPI_Receive(
  &hspi1, &rx, 1U, 2000U);
if (result == HAL_OK) {
 printf("RX=%02X\r\n",
        (unsigned int)rx);
}
```

수신 설정은 SPI_MODE_SLAVE와 SPI_NSS_HARD_INPUT을 사용한다. 외부 CS는 PA4로 입력된다. HAL_OK일 때만 수신값을 출력하며, 타임아웃과 다른 오류는 별도 메시지로 구분한다.

실제 결과와 해석



수신 모니터에서 RX=55, RX=A5, RX=00, RX=FF가 반복되는 것을 확인하였다. 따라서 MCU의 SPI 송신 바이트가 상대 MCU에 전달되고, 수신 결과가 VCP UART로 출력되는 통신 경로가 동작하였다.

실험 영상과 사진 모두 Github에 업로드하였다.
https://github.com/thechef030/STM32_3/tree/stm32-3/evidence/board

4. 과제 10 MCU → FPGA로 전송

설계 목표와 회로 구성

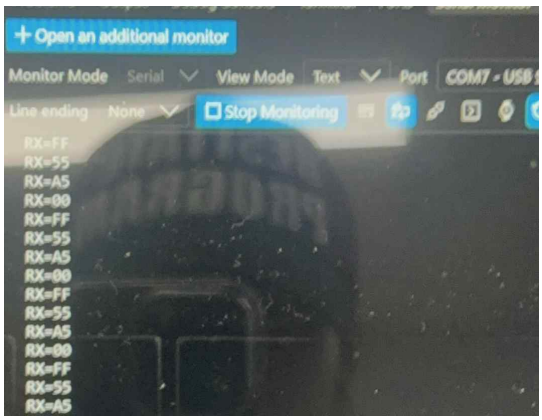
MCU의 SPI 송신 데이터를 FPGA가 수신하고, FPGA의 USB to Serial을 통해 PC에 표시하도록 구성하였다. MCU D3, D11, D10을 FPGA PORT A I/O 00, 01, 02에 연결하고 공통 GND를 연결하였다. FPGA에는 spi_rx_top의 비트스트림을 다운로드하였다.

FPGA 수신 회로 구조

spi_slave_rx는 SCK, MOSI, CS를 FPGA 내부 클록에 동기화하고 SCK 상승 에지를 검출한다. CS가 HIGH이면 비트 계수를 초기화하고, LOW이면 MOSI를 시프트 레지스터에 저장한다. 8번째 비트에서 완성된 바이트와 수신 유효 신호를 만든다. spi_uart_report와 uart_byte_tx가 이를 RX=XX 형식으로 출력한다.[2]

```
// 수신 동작을 요약한 의사 코드
if (CS == 1) bit_count = 0;
else if (SCK_rising) {
 shift = {shift[6:0], MOSI};
 if (bit_count == 7) {
  received_byte = shift;
  valid = 1;
 }
}
```

실제 결과와 예상값 비교



FPGA와 연결한 COM7의 모니터에 RX=FF, RX=55, RX=A5, RX=00이 순환하여 표시되었다. 캡처 시작점은 반복열의 중간이지만, FF 다음에 55가 나타나는 순환 관계가 송신 패턴과 일치한다. 한 번의 수신뿐 아니라 여러 주기의 반복 수신을 확인하였다. 이번 결과는 수신 FPGA 쪽 UART 로그가 송신 바이트와 일치하므로 MCU의 출력, 보드 간 배선, FPGA의 비트 수집, UART 출력까지 연결된 경로의 동작을 확인한 것이다.

실험 영상과 사진 모두 Github에 업로드하였다.
https://github.com/thechef030/STM32_3/tree/stm32-3/evidence/board

5. 과제 11 FPGA → MCU로 전송

설계 목표와 회로 구성

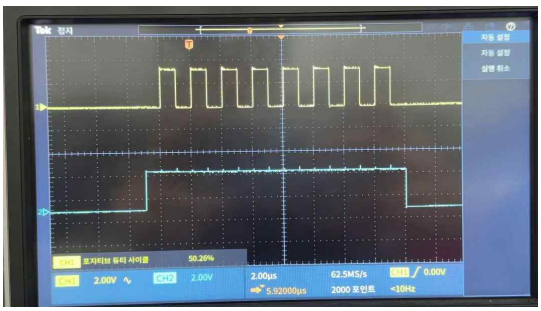
FPGA를 SPI 마스터 송신기로, MCU를 슬레이브 수신기로 설정하였다. FPGA I/O 00, 01, 02를 MCU D3, D11, A2에 연결하였다. 특히 MCU 수신 CS는 송신 때 사용한 D10이 아니라 A2이다. FPGA에는 spi_tx_top을 다운로드하고 MCU는 수신용 설정으로 실행하였다.

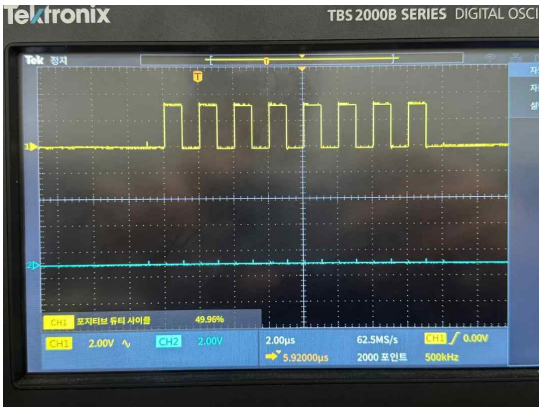
FPGA 송신 회로 구조

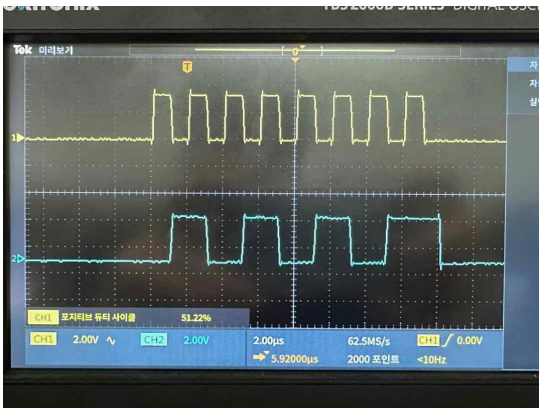
spi_master_tx는 내부 50 MHz 클록을 분주하여 500 kHz SCK를 만든다. CS를 LOW로 한 뒤 첫 비트를 준비하고, 상승 에지에서 데이터가 안정되도록 하강 에지마다 다음 비트로 이동한다. 8비트 전송 후 SCK를 LOW, CS를 HIGH로 복귀시킨다. 상위 회로는 55, A5, 00, FF를 순서대로 전달한다.

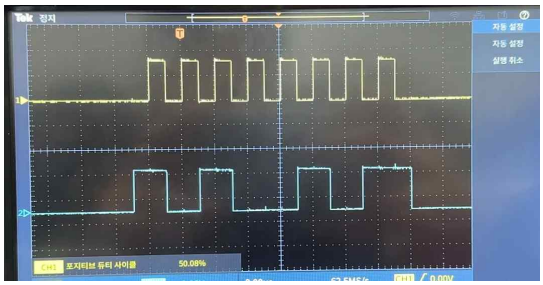
기본 분주값 HALF_CYCLES=50에서 SCK 주기는 2×50/50 MHz=2 μs이다. 8클록 구간은 설계상 16 μs이다. 이 값은 코드에서 계산한 값이며 별도의 커서 측정값과 구분한다.

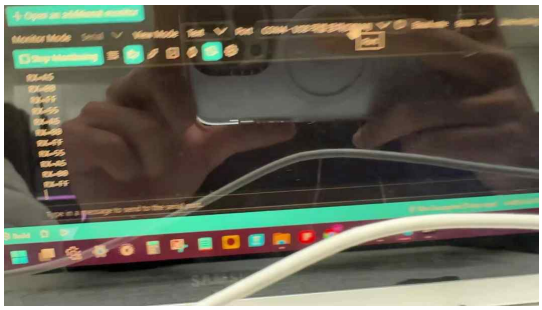
실제 결과와 해석











FPGA 송신 시 8개 SCK 펄스가 발생하고, 저장된 파형 구간에서는 MOSI가 HIGH로 유지되어 0xFF 패턴을 나타냈다. MCU 수신 모니터에서도 검증용 바이트가 반복되어 FPGA에서 MCU로의 전송 성공을 확인하였다. MCU 수신은 외부 SCK와 NSS를 사용하므로 송신 역할의 MCU 코드와 구분해야 한다.

실험 영상과 사진 모두 Github에 업로드하였다.

https://github.com/thechef030/STM32_3/tree/stm32-3/evidence/board

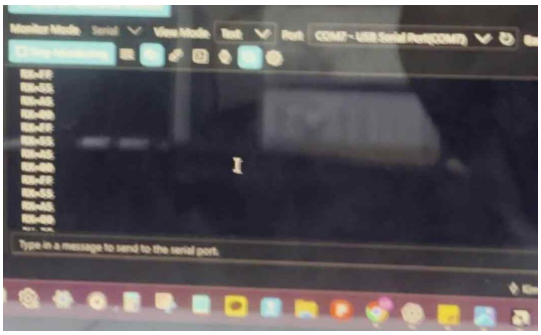
6. 과제 12 FPGA 간 SPI 통신

설계 목표와 회로 구성

두 FPGA 중 한 대에는 spi_tx_top, 다른 한 대에는 spi_rx_top의 비트스트림을 다운로드하였다. 송신 FPGA의 PORT A I/O 00, 01, 02를 수신 FPGA의 같은 번호에 연결하고 GND를 공통으로 하였다. 수신 FPGA의 USB to Serial을 PC와 연결하여 결과를 확인하였다.

두 보드에 같은 소스 파일을 추가할 수는 있으나 최상위 모듈에 따라 생성되는 회로가 달라진다. 송신 비트스트림과 수신 비트스트림을 각각 생성하고 해당 보드에 다운로드하였다. 송신 FPGA는 패턴을 자동 생성하므로 PC에서 문자를 입력하여 SPI 송신을 시작하는 방식은 아니다.

실제 결과와 비교



수신 PC의 모니터에 55, A5, 00, FF에 해당하는 RX 로그가 반복되었다. FPGA 송신 회로가 생성한 SPI 신호를 다른 FPGA가 수신하여 UART로 변환하는 전체 경로가 동작하였다. 동일한 검증 패턴을 사용하여 MCU가 포함된 과제 9~11과 결과를 비교할 수 있었다.

실험 영상과 사진 모두 Github에 업로드하였다.

https://github.com/thechef030/STM32_3/tree/stm32-3/evidence/board

7. DAC

설계 목표와 회로 구성

LAB6에서 완료하지 못했던 DAC 실험을 이어서 수행하였다. FPGA가 8비트 디지털 값을 증가시키고 병렬 DAC의 데이터·채널·쓰기 제어 신호를 출력하도록 구성하였다. 보드의 DAC 회로와 출력 표시부를 사용하여 값 변화에 따른 동작을 확인하였다.

핵심 코드와 예상 동작

```
// ramp8의 값 갱신 부분
assign update =
 (count == UPDATE_CYCLES-1);
// update 시 count 초기화 후
value <= value + 1'b1;

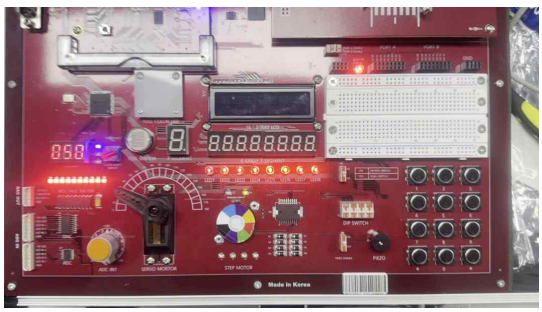
// dac_parallel8의 쓰기 부분
we_n <= 1;
if (update) begin
 data <= value;
 a_b <= channel;
 we_n <= 0;
end
```

기본 UPDATE_CYCLES=500000과 50 MHz 클록에서 갱신 간격은 10 ms이며, 8비트 값은 255 다음 0으로 순환한다. 전체 256단계는 설계상 약 2.56 s이다. 데이터 버스와 LED 출력은 같은 디지털 값을 나타내며, 실제 아날로그 전압은 DAC 회로에 의해 결정된다.

실제 결과와 비교



- DAC 시연 약 0.37 s의 보드 상태



- DAC 시연 약 2.24 s의 보드 상태. 표시부의 점등 범위가 달라진다.

이번 주 실험에서는 DAC 동작을 성공적으로 확인하였다. 시연 영상에서 디지털 LED 패턴과 전압 표시부의 점등 범위가 시간에 따라 달라졌다. 이는 값을 순차적으로 갱신하여 출력 상태를 변화시키려는 설계 목적에 부합한다. 영상은 보드 표시 변화를 확인하는 자료이며, 아날로그 출력의 최대·최소 전압이나 선형성 오차를 수치로 측정한 자료는 아니다. 따라서 전압 진폭과 오차율은 본 보고서에서 임의로 제시하지 않았다.

실험 영상과 사진 모두 Github에 업로드하였다.
https://github.com/thechef030/STM32_2/tree/stm32-2/evidence/Vivado

8. SERVO 제어

설계 목표와 회로 구성

LAB6에서 미완료한 SERVO 제어를 이어서 수행하였다. servo_position에서 버튼 입력에 따라 목표 펄스 폭을 변경하고, servo_pwm에서 주기적인 펄스를 만들어 보드의 서보모터 제어 신호로 출력하였다. 두 모듈은 servo_ctrl에서 연결된다.

핵심 동작과 예상 결과

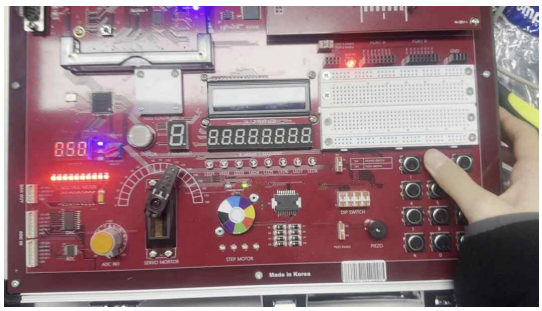
```
// 위치 갱신 동작을 요약한 의사 코드
reset: pulse_us = 1500;
left_only:  pulse_us -= 100;
right_only: pulse_us += 100;
limit: 1000 <= pulse_us <= 2000;
both: keep_previous;
// PWM 프레임 시작에서 새 폭 적용
frame_period = 20 ms;
```

설계상 PWM 주기는 20 ms로 50 Hz이며, HIGH 폭은 1.0~2.0 ms 범위에서 0.1 ms씩 변한다. 초기 폭은 1.5 ms이고 동시에 두 입력이 들어오면 이전 위치를 유지한다. 새 폭은 다음 프레임부터 적용하여 진행 중인 펄스의 폭이 갑자기 바뀌지 않도록 한다.

실제 결과와 비교



- SERVO 시연 약 3.94 s의 모터 암 위치



- SERVO 시연 약 6.76 s의 모터 암 위치

이전 화면과 회전 방향이 달라진다.

버튼 조작에 따라 서보모터 암의 위치가 변하는 것을 확인하였다. 두 시점의 모터 암 방향이 서로 다르고 영상에서도 회전 과정이 나타나므로, 이번 주에는 위치 제어 동작을 성공적으로 확인하였다. LAB6의 미완료 항목을 후속 실험에서 완료한 결과이다.

1.0~2.0 ms와 20 ms는 설계 설정값이다. 이번 시연은 기계적 회전을 보여 주지만, 실제 펄스 폭과 회전 각도를 수치로 측정한 자료는 없으므로 특정 각도나 각도 오차까지 검증한 것으로 서술하지 않았다.

실험 영상과 사진 모두 Github에 업로드하였다.

https://github.com/thechef030/STM32_2/tree/stm32-2/evidence/Vivado

9. 소스와 Github주소

소스와 실험 증거는 아래 GitHub 링크에서 확인할 수 있다.

- Github 주소:

https://github.com/thechef030/STM32_3

- 전체 커밋 해시:

a582a795a8831bf6ab85

846893601eacf95ae3a3

- 태그: v1.1.0

- DAC와 SERVO 코드 및 영상 주소:

https://github.com/thechef030/STM32_2

- 전체 커밋 해시:

3396237b7c8511caa3d0

a9079f9412f6fdda9799

10. 참고문헌

전전설2 LAB7 교안